Our current rule-based detector is a **multi-indicator, weighted evidence system**. It does **not** say “high entropy = DNS tunnel.” It checks five separate indicators on each DNS window and combines the triggered indicators into a score.

What it uses

For every unique window:

```
(src_ip, domain, 60-second window)
```

we first calculate the 19 features. The rule detector currently uses **5 of those features**:

Rule	Feature used	Current threshold	What it indicates
Long Subdomain	`max_subdomain_length`	`>= 50`	Unusually long subdomain
High Subdomain Entropy	`avg_subdomain_entropy`	`>= 3.5`	Random/encoded-looking subdomains
High Query Rate	`queries_per_second`	`>= 1.0`	High DNS request frequency
High Unique Subdomain Ratio	`unique_subdomain_ratio`	`>= 0.8`	Many queries use different subdomains

High NXDOMAIN Rate	`nxdomain_rate`	`>= 0.3`	Large fraction of queries fail

These are currently defined in `config.py`.

---

1. Long subdomain

We calculate:

```
max_subdomain_length
```

For example:

```
abc.google.com
^^^^^^^^^
subdomain = abc
length = 3
```

versus:

```
a8f93jf8sdf98sdf98sdf98sdf98sdf98sdf.example.com
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

The rule triggers when:

```
max_subdomain_length >= 50
```

So:

```
42 → no
49 → no
50 → YES
75 → YES
```

The idea is that DNS tunneling can encode data into long labels.

---

2. High subdomain entropy

We calculate **Shannon entropy** of each subdomain and then take the average:

```
avg_subdomain_entropy
```

Our entropy function essentially calculates:

H(X)=−∑p(x)log2p(x)H(X)=-\sum p(x)\log_2p(x)

A repetitive/readable string tends to have lower entropy.

For example, something like:

```
aaaaaaaaaa
```

has very low entropy.

Something more varied like:

```
a8F3kP91xZ7q
```

has higher entropy.

Our current rule:

```
avg_subdomain_entropy >= 3.5
```

→ triggered.

Important: this does NOT mean entropy ≥ 3.5 proves tunneling. It is only one piece of evidence.

---

3. High query rate

We calculate:

```
queries_per_second
```

from:

```
number of queries / 60-second window
```

Because our current window is 60 seconds:

```
30 queries → 0.5 q/s
60 queries → 1.0 q/s
120 queries → 2.0 q/s
```

Current rule:

```
queries_per_second >= 1.0
```

→ triggered.

This captures the possibility of a DNS channel repeatedly sending encoded chunks.

---

4. High unique-subdomain ratio

This one is particularly important.

We calculate:

unique subdomain ratio=unique subdomainsqueries containing subdomains\text{unique subdomain ratio} = \frac{\text{unique subdomains}} {\text{queries containing subdomains}}

Example:

```
abc.example.com
abc.example.com
abc.example.com
def.example.com
```

There are:

```
4 queries
2 unique subdomains
```

so:

```
ratio = 2/4 = 0.5
```

But:

```
a1.example.com
a2.example.com
a3.example.com
a4.example.com
a5.example.com
```

gives:

```
5 unique / 5 queries = 1.0
```

Our threshold:

```
unique_subdomain_ratio >= 0.8
```

→ triggered.

This is useful because a tunnel may generate a stream of changing labels carrying different chunks of data.

But we already discovered a problem here

With only **one query**:

```
abc.example.com
```

we get:

```
unique_subdomain_count = 1
query_count = 1

ratio = 1/1 = 1.0
```

That would falsely trigger the rule.

That's why we added:

```
MINIMUM_QUERY_COUNT = 10
```

The rules are evaluated, but they **cannot contribute to the detection score unless at least 10 queries exist in the window**.

That's an important safeguard.

---

5. NXDOMAIN rate

We calculate:

NXDOMAIN rate=NXDOMAIN queriestotal queries\text{NXDOMAIN rate} = \frac{\text{NXDOMAIN queries}} {\text{total queries}}

Example:

```
100 DNS queries
30 NXDOMAIN
```

gives:

```
NXDOMAIN rate = 0.30
```

Our current threshold:

```
nxdomain_rate >= 0.3
```

→ triggered.

A high NXDOMAIN rate can be an indicator of suspicious DNS behavior, although legitimate software can also generate NXDOMAINs.

How the five rules become one score

Currently every rule has weight:

```
RULE_WEIGHTS = {
    "long_subdomain": 1.0,
    "high_subdomain_entropy": 1.0,
    "high_query_rate": 1.0,
    "high_unique_subdomain_ratio": 1.0,
    "high_nxdomain_rate": 1.0,
}
```

So maximum score:

5.05.0

If a window triggers:

```
Long Subdomain
High Query Rate
High Unique Subdomain Ratio
```

then:

```
score = 1 + 1 + 1 = 3
```

and:

```
normalized_score = 3 / 5 = 0.6
```

Current detector output looks like:

```
Query count: 35

Evidence: MET
```

```
Triggered rules:
    long_subdomain
    high_query_rate
    high_unique_subdomain_ratio

Score: 3.0 / 5.0

Normalized score: 0.60
```

The minimum-evidence logic

This is very important.

The detector first checks:

```
query_count >= MINIMUM_QUERY_COUNT
```

where:

```
MINIMUM_QUERY_COUNT = 10
```

Therefore:

5 queries

```
Rules may individually look triggered
        ↓
BUT
        ↓
minimum evidence NOT met
        ↓
score = 0
```

10+ queries

```
minimum evidence MET
        ↓
triggered rules contribute to score
```

So the system isn't simply:

```
if entropy > threshold:
    TUNNEL
```

It's currently:

```
DNS window
    ↓
calculate features
    ↓
check 5 indicators
    ↓
require ≥10 queries
    ↓
sum weights of triggered indicators
    ↓
score / maximum score
```

One important thing: we have NOT defined "tunnel = score X"

This is intentional.

Right now:

```
0/5
1/5
2/5
3/5
4/5
5/5
```

are **evidence scores**, not final classifications.

We haven't yet said:

```
0–1 = normal
2–3 = suspicious
4–5 = tunnel
```

because that would be arbitrary without testing.

Our next phase should be to generate **controlled normal DNS traffic and controlled tunnel-like/synthetic DNS traffic**, observe these five indicators, and then determine whether these thresholds and weights actually separate the two behaviors.

That is where our rule-based detector becomes experimentally justified rather than just a collection of plausible thresholds.